

Predicción de Churn de Clientes

Resumen completo del proyecto

Equipo: Bautista Rios · Tomás Attas · Agustín Venutolo · Francisco Cavanna · Agustin Picciolo

Cátedra: Inteligencia Artificial Aplicada a Negocios — Licenciatura en Negocios y Tecnología

Cuatrimestre: Primer cuatrimestre 2026 **Fecha de este documento:** Junio 2026

1. Resumen ejecutivo

Construimos un sistema de predicción de churn para un e-commerce con **5,630 clientes activos y 17% de churn anual**. La pregunta del gerente comercial fue doble: **detectar a quiénes están por irse antes de que se vayan, y entender por qué nos dejan**.

El proyecto se estructuró en seis fases: EDA con hipótesis de negocio, investigación de anomalías, feature engineering, preparación de datos, modelado, y comunicación. **El modelo final (Random Forest tuneado) detecta el 95.3% de los churners reales en test set**, con un trade-off explícito hacia Recall sobre Precision alineado con el costo asimétrico de perder un cliente versus enviarle un email innecesario.

Se documentaron **19 decisiones formales** y se identificaron **4 hallazgos accionables** que dieron lugar a **4 recomendaciones de negocio priorizadas**. El proceso incluyó refutaciones empíricas de creencias del equipo comercial (la regla "email a 15 días sin compra" no se sostiene) y validación cuantitativa de feature engineering (las 2 features adoptadas terminaron #1 y #3 en feature importance del modelo final).

2. Contexto del proyecto

2.1 Problema de negocio

El año pasado el 17% de la base de clientes (945 sobre 5,630) dejó de comprarnos. En términos económicos:

- **Adquirir un cliente nuevo cuesta entre 5 y 7 veces más que retener uno existente** (estándar de la industria).
- **Los churners que no detectamos son costos de adquisición que tenemos que afrontar** para volver al mismo volumen de base.
- **Un cliente con 3+ meses de actividad** típicamente vale más en los siguientes 12 meses que uno recién adquirido.

2.2 El dataset

- **Origen:** `data/raw/E_Commerce_Dataset.csv` (Kaggle, público)
- **Filas:** 5,630 clientes
- **Columnas:** 20 (1 ID + 1 target + 18 features)
- **Target:** **Churn** (binaria, 1 = se fue / 0 = sigue activo)

- **Desbalance: 16.8% positivo / 83.2% negativo** → clase minoritaria es la que queremos detectar
- **Nulos:** 7 columnas con 4-5% de nulos cada una
- **Duplicidad de categorías:** 3 columnas con etiquetas escritas de varias formas (CC vs Credit Card, etc.)
— unificadas en pre-procesamiento

Variables principales:

Tipo	Variables
Identificador	CustomerID
Target	Churn
Comportamiento	Tenure, DaySinceLastOrder, OrderCount, OrderAmountHikeFromlastYear
Financieras	CashbackAmount, CouponUsed
Engagement	HourSpendOnApp, NumberOfDeviceRegistered
Demográficas	Gender, MaritalStatus, CityTier
Satisfacción	SatisfactionScore (1-5), Complain (binaria)
Comerciales	PreferredPaymentMode, PreferredOrderCat, PreferredLoginDevice
Geográficas	WarehouseToHome, NumberOfAddress

3. EDA — Las 6 hipótesis originales

Antes de modelar formulamos seis hipótesis de negocio que pudieran guiar campañas de retención. Cada una se testeó con un test estadístico apropiado y se interpretó en términos de negocio.

H1 — Clientes nuevos (Tenure bajo) tienen mayor riesgo

- **Lógica de negocio:** sin hábito de compra formado, es más fácil probar alternativas. Los primeros 6 meses son críticos para "enganchar" al cliente.
- **Test estadístico:** Mann-Whitney U (comparación de distribuciones no paramétrica)
- **Resultado:** Tenure promedio = **3.4 meses** (churn) vs **11.5 meses** (activos), $p < 1e-6$
- **Status:** ✓ **CONFIRMADA**
- **Acción sugerida:** programa estructurado de onboarding en los primeros 6 meses

H2 — Clientes que se quejaron (Complain=1) churnean más

- **Lógica de negocio:** una queja es señal directa de insatisfacción. Si no se resuelve, el cliente se va.
- **Test estadístico:** Chi-cuadrado (variable categórica vs target)
- **Resultado:** $\chi^2 = 350.9$, $p < 1e-6$ (asociación muy fuerte)
- **Status:** ✓ **CONFIRMADA** ⚠ con alerta de leakage

- **⚠ Riesgo de leakage temporal:** no podemos confirmar si la queja se registra antes o después de que el cliente decide irse. Si se registra después, no estaría disponible al momento de predecir.

H3 — Mayor inactividad (DSL alto) predice churn

- **Lógica de negocio:** un cliente que no compra hace 15+ días en un e-commerce probablemente encontró una alternativa.
- **Test estadístico:** Mann-Whitney U
- **Resultado: REFUTADA con relación INVERTIDA.** Los churneados ordenaron **MÁS** recientemente (mediana 2.0 días) que los activos (mediana 4.0 días), $p < 1e-42$ en la dirección opuesta.
- **Status: ✗ REFUTADA → motivó investigación profunda** (sección 4)
- **Acción descartada:** la regla histórica "email de reactivación a los 15 días sin compra" no se sostiene empíricamente con esta base.

H4 — Menor satisfacción (SatisfactionScore bajo) predice churn

- **Lógica de negocio:** la hipótesis más intuitiva — cliente insatisfecho tiene motivos para irse.
- **Test estadístico:** Mann-Whitney U
- **Resultado: CONTRAINTUITIVO.** Los churneados reportan score promedio **MÁS ALTO** (3.39 vs 3.00 activos), $p < 1e-15$.
- **Status: ⚠ CONTRAINTUITIVO → motivó investigación profunda** (sección 4)

H5 — Menor cashback predice churn

- **Lógica de negocio:** el cashback actúa como incentivo financiero para quedarse. Clientes con poco cashback tienen menor "costo de salida" percibido.
- **Test estadístico:** Mann-Whitney U
- **Resultado:** Cashback promedio = **\$160** (churn) vs **\$181** (activos), $p < 1e-6$
- **Status: ✓ CONFIRMADA**
- **Acción sugerida:** aumentar el cashback como palanca de retención para clientes de alto riesgo identificados por el modelo.

H6 — Clientes Single churnean más que casados y divorciados

- **Lógica de negocio:** los solteros suelen tener menor "anclaje" al servicio — compras más impulsivas, menos recurrentes para hogar, menor costo de cambio percibido.
- **Test estadístico:** Chi-cuadrado
- **Resultado:** Single **26.7%** churn ($n=1,796$) vs Divorced 14.6% ($n=848$) vs Married 11.5% ($n=2,986$). $\chi^2 = 188.7$, $p \approx 1e-41$.
- **Status: ✓ CONFIRMADA — fuerte.** Los Single casi duplican el promedio general (16.8%).
- **Acción sugerida:** segmentación específica del segmento Single en campañas de fidelización.

Resumen de las 6 hipótesis

#	Variable	Test	Resultado	Status
---	----------	------	-----------	--------

H1	Tenure	Mann-Whitney	3.4 vs 11.5 meses, $p < 1e-6$	✓ Confirmada
H2	Complain	Chi-cuadrado	$\chi^2=350.9$, $p < 1e-6$	✓ Confirmada $\triangle$ leakage
H3	DaySinceLastOrder	Mann-Whitney	Relación INVERTIDA	✗ Refutada → investigada
H4	SatisfactionScore	Mann-Whitney	Score MÁS ALTO en churn	$\triangle$ Contraintuitiva → investigada
H5	CashbackAmount	Mann-Whitney	\$160 vs \$181, $p < 1e-6$	✓ Confirmada
H6	MaritalStatus	Chi-cuadrado	Single 26.7%, Married 11.5%	✓ Confirmada (fuerte)

4. Investigación de las anomalías H3 y H4

Las dos hipótesis con resultados raros (H3 refutada con inversión, H4 contraintuitiva) **no se aceptaron a ciegas**. Se hizo un notebook de seguimiento ([01b_Investigacion_Anomalias_H3_H4.ipynb](#)) con tres sub-hipótesis explicativas para cada una, validadas empíricamente.

4.1 Investigación de H3 — ¿Por qué la inactividad NO predice churn?

H3a — ¿El resultado es un sesgo por nulos?

- **Hipótesis:** si los churneados tienen más nulos en `DaySinceLastOrder`, el `.dropna()` del EDA recorta selectivamente la cola "muchos días sin comprar".
- **Test:** porcentaje de nulos por grupo + Chi-cuadrado
- **Resultado:** 5.4% nulos en activos vs 5.7% en churn. χ^2 p = **0.78** (sin diferencia significativa)
- **Conclusión:** ✗ **Refutada** — los nulos NO sesgan el resultado.

H3b — ¿Tenure es un confundidor?

- **Hipótesis:** los clientes nuevos (Tenure bajo) son los que más churnean y por construcción tienen `DSL` bajo (acaban de registrarse y ordenar). La inversión podría ser consecuencia de H1.
- **Test:** correlación Spearman entre Tenure y DSL + análisis estratificado de churn por `DSL` × Tenure
- **Resultado:** Spearman ρ = **0.205**, $p < 1e-49$. Estratificando por cuantiles de Tenure: en el Q1 (más nuevos) el churn es **38-59%** para TODOS los cuantiles de DSL.
- **Conclusión:** ✓ **CONFIRMADA** — Tenure es confundidor. La inversión es producto de mezclar perfiles de edad de cliente distintos.

H3c — ¿La mala experiencia reciente explica el patrón invertido?

- **Hipótesis:** si los clientes con DSL bajo + Complain=1 son los que más churnean, entonces "compra reciente + queja" es el patrón que arrastra la mediana.
- **Test:** tasa de churn por intersección `DSL` × Complain
- **Resultado:**

--	--	--	--	--

Complain	DSL	Churn
Sin queja	Lejano	7.1%
Sin queja	Reciente	14.2%
Con queja	Lejano	23.5%
Con queja	Reciente	38.9%

- **Conclusión:** ✓ **CONFIRMADA**. El verdadero patrón de riesgo no es la inactividad sostenida — es **"compra reciente + queja sin resolver"** (5.5× la base).

Síntesis H3

La relación invertida del EDA original es **real pero engañosa**. Tiene dos causas que se solapan:

1. Los clientes nuevos (Tenure bajo) dominan la población churneada y arrastran la mediana de DSL hacia abajo (confundidor).
2. El patrón causal real es $DSL \times Complain$, no DSL solo.

Implicancia de negocio: la regla histórica "email de reactivación a los 15 días sin compra" se retira formalmente. La nueva acción de negocio recomendada: alertas sobre quejas sin resolver en clientes recientes.

4.2 Investigación de H4 — ¿Por qué los satisfechos churnean más?

H4a — ¿La escala del score está invertida?

- **Hipótesis:** si el dataset codifica 1 = muy satisfecho y 5 = muy insatisfecho (al revés del rótulo), el resultado del EDA estaría reportando "los insatisfechos churnean más" — coherente con la hipótesis.
- **Test:** comparar score promedio en clientes con vs sin queja. Si la escala está invertida, los con queja deberían tener score MÁS ALTO.
- **Resultado:** score con queja = 2.999; sin queja = 3.094. Diferencia = **-0.095** (casi nula).
- **Conclusión:** ✗ **Refutada** — la escala no está invertida. Pero curiosamente el score y la queja son **casi independientes** entre sí.

H4b — ¿Tenure es confundidor parcial?

- **Hipótesis:** los clientes nuevos tienden a calificar más alto (efecto novedad) y a churnear más. Eso infla el score promedio del grupo churneado.
- **Test:** stratificación por cuantiles de Tenure
- **Resultado:** el patrón "score alto → más churn" es **fuerte en el Q1 (nuevos)** — sube de 36% a 57% al subir el score — pero **se aplana en clientes viejos** (Q4 ≈ 4-5% en todos los scores).
- **Conclusión:** ⚠ **PARCIAL** — Tenure confunde el resultado pero la dirección "score alto → más churn" es real en el segmento de mayor masa de churners.

H4c — ¿El score tiene bajo poder predictivo?

- **Test:** información mutua con el target, comparada con otras features conocidas.

- **Resultado:**

Feature	Información mutua
Tenure	0.1359
CashbackAmount	0.0344
Complain	0.0275
DaySinceLastOrder	0.0262
SatisfactionScore	0.0054

- **Conclusión:** ✓ **CONFIRMADA.** El score tiene **25× menos información mutua que Tenure**. Es predictor débil — el churn rate sube con el score (11.5% → 23.8%) pero la varianza de la relación es alta. La hipótesis: el score auto-reportado no refleja la verdadera intención de irse (puede estar inflado, gameado, o medido en otro momento del journey del cliente).

Síntesis H4

El score de satisfacción tiene una **relación monótona real con churn** (más alto → más churn) pero **el poder predictivo es muy bajo**. No usar como gatillo de campañas de retención. Mantener en el modelo solo si aporta marginalmente.

5. Pipeline de preparación de datos (anti-leakage)

El pipeline está implementado en `src/preprocessing.py` con disciplina **fit-on-train**: todas las estadísticas (medianas, percentiles, encoders) se calculan SOLO con train y se aplican a test.

Flujo:

```
raw CSV
↓ clean_categories (sin estadísticas – seguro pre-split)
↓ dataset_limpio.csv
↓ split estratificado (test_size=0.2, random_state=42)
↓ apply_impute (mediana fit-on-train, 7 columnas con nulos)
↓ apply_caps (percentil 99 fit-on-train, 3 columnas)
↓ add_features (row-wise, 6 features derivadas)
↓ OneHotEncoder (fit-on-train, 5 nominales → 17 cols)
↓
4 CSVs procesados en data/processed/
```

Decisiones del pipeline (todas en `decisions.md`):

Paso	Decisión	Por qué
Limpieza	Unificar categorías duplicadas (CC/ Credit Card, etc.)	Si no, el modelo trata duplicados como distintos y genera OHE redundantes

Split	<code>stratify=y, test_size=0.2, random_state=42</code>	Con 17% churn, split aleatorio puede dejar test y train con distribuciones distintas
Orden	Split ANTES de imputar	Para que las medianas no usen información del test
Imputación	Mediana (no media)	Robusta a outliers detectados en el EDA
Outliers	Cap al percentil 99 (no eliminar)	Reduce ruido sin perder clientes; $Tenure \leq 30$, $WarehouseToHome \leq 35$, $NumberOfAddress \leq 11$
Encoding	One-Hot (no LabelEncoder)	Las 5 categóricas son nominales; LabelEncoder impondría orden falso
Complain	Guardar 4 CSVs (con y sin)	Para auditar empíricamente el riesgo de leakage

Audit de la variable **Complain**

Como no podemos confirmar el timing de la queja vs el churn (dataset público), se hizo un audit cuantitativo: re-entrenar el modelo ganador sobre `con_complain.csv` y medir el lift.

Regla escrita ANTES del audit (`decisions.md`):

- Gap Recall ≤ 0.05 → Complain aporta poco, drop confirmado
- Gap $0.05 - 0.15$ → aporta pero no validable → mantener drop (conservador)
- Gap > 0.15 → sospecha fuerte de leakage → drop confirmado

Resultado del audit:

Métrica (CV)	Sin Complain	Con Complain	Gap
Recall	0.8431	0.8509	+0.0079
PR-AUC	0.9094	0.9289	+0.0195

Decisión final: ✓ **DROP DEFINITIVO de Complain**. Aporta menos de 1% de Recall — no compensa el riesgo de leakage no validable.

6. Feature Engineering

6.1 Features originales (4, desde el pipeline base)

Derivadas de las hipótesis confirmadas:

Feature	Fórmula	Hipótesis
<code>CashbackPerOrder</code>	<code>CashbackAmount / OrderCount</code>	H5 (incentivo financiero)
<code>CouponPerOrder</code>	<code>CouponUsed / OrderCount</code>	intensidad de uso

AppHoursPerDevice	HourSpendOnApp / NumberOfDeviceRegistered	engagement
IsNewCustomer	(Tenure ≤ 3).astype(int)	H1 (riesgo temprano)

6.2 Exploración de 7 candidatas adicionales

Se evaluaron 7 candidatas adicionales con criterio cuantitativo escrito **antes** de ver los resultados. Cada una pasó por 3 puertas:

1. **Gate de información:** Información mutua (MI) con **Churn** ≥ 0.005
2. **Gate de redundancia:** Máxima correlación con features existentes ≤ 0.85
3. **Lift en Random Forest:** Mejora del Recall en CV 5-fold

#	Candidata	Origen	MI	Max corr	Lift Recall	Decisión
1	RecentPurchaseWithComplaint = (DSL≤3) & (Complain==1)	H3 (01b)	0.028 ✓	0.695 ✓	-0.0039	✗
2	NewCustomerComplaint = (Tenure≤3) & (Complain==1)	H1+H2	0.064 ✓	0.513 ✓	-0.0066	✗
3	OrdersPerMonth = OrderCount / (Tenure+1)	/ml-churn	0.118 ✓	0.529 ✓	+0.0106	✓ ADOPTAR
4	CashbackPerMonth = CashbackAmount / (Tenure+1)	/ml-churn	0.148 ✓	0.843 ✓	+0.0053	✓ ADOPTAR
5	HighSatisfaction = (Score≥4)	H4 (01b)	0.003 ✗	0.831 ✓	+0.0027	✗ falla gate MI
6	MultiAddress = (NumberOfAddress≥5)	/ml-churn-style	0.000 ✗	0.851 ✗	-0.0026	✗ falla ambos
7	Dormant = (DaySinceLastOrder≥14)	H3 (01b)	0.000 ✗	0.411 ✓	-0.0026	✗ falla gate MI

6.3 Adopción final: 2 features

Solo dos candidatas pasaron las tres puertas:

- **OrdersPerMonth** = OrderCount / (Tenure + 1)
- **CashbackPerMonth** = CashbackAmount / (Tenure + 1)

Ambas capturan la **tasa por mes** de actividad relativa a la antigüedad del cliente — una señal que las variables crudas mezclan con la edad de la relación.

6.4 Hallazgo no obvio

Las dos interacciones explícitas (RecentPurchaseWithComplaint, NewCustomerComplaint) **pasan los gates**

de evidencia (MI, Spearman, redundancia) pero degradan el Recall del Random Forest. Esto evidencia operativamente que el RF **descubre interacciones de 2 vías nativamente** vía splits anidados — agregárselas pre-computadas solo introduce varianza.

6.5 Validación retrospectiva

Las dos features adoptadas terminaron **#1 y #3 en feature importance del modelo final**:

#	Feature	Importance
1	CashbackPerMonth	0.147
2	Tenure (raw)	0.103
3	OrdersPerMonth	0.101

El protocolo de exploración + benchmark funcionó.

7. Modelado

7.1 Setup del experimento

Criterio escrito **antes** de ver resultados (en `decisions.md` #1 y notebook 03 sección 4):

```
Métrica primaria (optimización CV): Recall (clase 1 = Churn)
Justificación: costo asimétrico — perder un churner cuesta varias veces
                más que un email innecesario.

Métricas reportadas: F1, Recall, Precision, PR-AUC, AUC-ROC

Cross-validation: StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

Baseline obligatorio: DummyClassifier(strategy='most_frequent')

Threshold de aceptación: Recall (CV) > Dummy + 0.05; gap train-test ≤ 0.05

Criterio del ganador: máximo Recall CV; desempate por PR-AUC

Tuning: top 2 modelos con BayesSearchCV (n_iter=30); adoptar tuneado
        solo si mejora Recall ≥ 2% vs defaults.
```

7.2 Modelos comparados

Modelo	Hiperparámetros iniciales	Justificación
DummyClassifier	<code>strategy='most_frequent'</code>	Piso obligatorio per skill /ds-model y rúbrica
DecisionTree	<code>class_weight='balanced', max_depth=10, random_state=42</code>	Obligatorio per rúbrica — interpretable como reglas if/then

RandomForest	<code>n_estimators=200, class_weight='balanced', random_state=42</code>	Captura interacciones nativamente, robusto
XGBoost	<code>n_estimators=200, scale_pos_weight=4.94, random_state=42</code>	Esperado el mejor en tabular desbalanceado

7.3 Resultados de CV inicial

Modelo	Recall CV (mean ± std)	Precision CV	F1 CV	PR-AUC CV	AUC-ROC CV
DummyClassifier	0.000 ± 0.000	0.000	0.000	0.168	0.500
DecisionTree	0.765 ± 0.038	0.601	0.673	0.667	0.858
RandomForest	0.843 ± 0.032	0.829	0.835	0.909	0.970
XGBoost	0.834 ± 0.043	0.884	0.858	0.901	0.966

Familia ganadora: RandomForest (`decisions.md` #17). Razones:

- Mejor Recall CV
- Menor varianza entre folds (std 0.032 vs 0.043 de XGBoost)
- Gana también el desempate por PR-AUC

7.4 Tuning iterativo (3 rondas con BayesSearchCV)

Ronda 1 — Original (n_iter=30, ranges iniciales)

- RF tuneado: Recall **0.8589** → **+1.88%** vs defaults
- XGB tuneado: Recall **0.8470** → **+1.58%** vs defaults
- **Ambos quedaron debajo del 2% threshold → conclusión inicial: descartar tuneado**
- **PERO una auditoría post-mortem detectó que 3/4 best_params de RF y 2/5 de XGB estaban pegados a los límites del search space.** El optimizer quería ir más allá pero no podía.

Ronda 2 — V1 expanded (n_iter=100, ranges expandidos)

- RF tuneado: Recall **0.8629** → **+2.34%** ✓ supera 2%
- XGB tuneado: Recall **0.8536** → **+2.38%** ✓ supera 2%
- Sin embargo, XGB dio config sospechosa: `n_estimators=50` (al piso del rango) — local optimum.

Ronda 3 — V2 narrow targeted (n_iter=50)

- RF: **0.8615** (plateau confirmado en iter 27)
- XGB: **0.8562** (config se estabilizó en `n_estimators=277`, típico)
- V2 confirma V1 en RF — resultado estable, no es ruido.

7.5 Análisis de overfitting (gap train-CV)

Para validar que los tuneados no overfitteen, se midió la diferencia entre Recall en train vs CV de las 6 configs:

Config	Recall train	Recall CV	Gap	Verdict
RF default	1.000	0.843	+0.157	Línea base
RF V0 (n_iter=30)	0.999	0.859	+0.140	Mejor que default
RF V1 (n_iter=100)	1.000	0.863	+0.137	Gana en ambas dimensiones
RF V2 (n_iter=50)	1.000	0.862	+0.139	Cerca de V1
XGB default	1.000	0.834	+0.166	—
XGB V2	1.000	0.856	+0.144	Mejor que XGB default

Hallazgo: TODOS los tuneados overfittean MENOS que sus defaults. El miedo a `max_depth` alto no se materializó — el ensemble compensa.

7.6 Ganador final: RandomForest V1 tuneado

```
RandomForestClassifier(  
    n_estimators=1469,  
    max_depth=50,  
    min_samples_leaf=3,  
    max_features=0.978,  
    class_weight='balanced',  
    random_state=42,  
)
```

7.7 Performance en test set

Test set evaluado dos veces — primero con RF defaults (resultados de la primera ronda), después con RF V1 tras la iteración del tuning. Declarado como limitación.

Métrica	RF defaults (1ª eval)	RF V1 (2ª eval, ganador final)	Δ
Recall	0.9421	0.9526	+0.0105 ✓
Precision	0.9179	0.8117	-0.1062 ⚠
F1	0.9299	0.8765	-0.0534
PR-AUC	0.9839	0.9611	-0.0228
AUC-ROC	0.9964	0.9917	-0.0047

Matriz de confusión del RF V1 en test (1126 clientes):

	Predicho: Activo	Predicho: Churn
Real: Activo (n=936)	TN = 894	FP = 42

Real: Churn (n=190)	FN = 9	TP = 181
---------------------	--------	----------

Lectura de negocio: sobre 190 churners reales, el modelo detectó **181 (Recall 95.3%)**. Generó 42 falsas alarmas sobre 936 clientes activos (4.5% extra de "ruido") — trade-off Recall↑/Precision↓ alineado con el costo asimétrico documentado en la decisión #1.

7.8 Interpretabilidad — SHAP

Análisis SHAP global (qué features pesan más en promedio) y local (descomposición de predicciones individuales). Los 2 ejemplos locales analizados:

- **True Positive** de alta probabilidad — el modelo acertó al marcar a un churner real
- **False Negative** de baja probabilidad — el modelo dijo "se queda" pero el cliente se fue (caso "perdido")

Top 10 features por importance built-in:

#	Feature	Importance
1	CashbackPerMonth (PR #4)	0.147
2	Tenure	0.103
3	OrdersPerMonth (PR #4)	0.101
4	CashbackAmount	0.063
5	IsNewCustomer	0.052
6	CashbackPerOrder	0.051
7	WarehouseToHome	0.051
8	DaySinceLastOrder	0.045
9	NumberOfAddress	0.042
10	OrderAmountHikeFromlastYear	0.038

8. Las 19 decisiones documentadas

Todas en `decisions.md`. Cada una sigue el template del skill `/add-decision` (Qué decidí / Por qué / Alternativas / Consecuencias).

#	Decisión
1	Elección de métrica principal — Recall
2	Tratamiento de la variable Complain (inicial)
3	Unificación de categorías inconsistentes

4	Manejo de outliers (Tenure=60, WarehouseToHome=126)
5	Split estratificado train/test
6	Orden del pipeline: split ANTES de imputar (evitar leakage)
7	Imputación de nulos con mediana
8	Cap de outliers al percentil 99 (resuelve decisión #4)
9	One-Hot Encoding para categóricas nominales
10	Feature engineering original (4 features derivadas)
11	Dos versiones de la base (con y sin Complain)
12	Estructura: preparación separada del modelado
13	Resolución H3 — DaySinceLastOrder se mantiene como feature
14	Resolución H4 — SatisfactionScore queda en evaluación
15	Adopción de features per-tenure (OrdersPerMonth, CashbackPerMonth)
16	Descarte de 5 candidatas de feature engineering
17	Random Forest como familia ganadora
18	Adopción final de Random Forest V1 tuneado tras iteración del tuning
19	Drop definitivo de Complain por riesgo de leakage no validable

9. Hallazgos accionables de negocio

Cuatro hallazgos clave para el equipo comercial, derivados del modelo + investigaciones:

Hallazgo 1 — La tasa mensual de actividad es el predictor más fuerte

Un cliente con pocas órdenes y poco cashback recibido **por mes de antigüedad** tiene riesgo alto. Esto es distinto a "pocas órdenes" en absoluto — un cliente nuevo con pocas órdenes puede ser saludable; uno con un año y pocas órdenes está claramente desenganchado.

Implicancia: la métrica para identificar riesgo no es "cuándo fue la última compra" sino "qué tan activo está este cliente para los meses que lleva con nosotros".

Hallazgo 2 — Los primeros 3 meses concentran el riesgo

Clientes con ≤ 3 meses tienen tasa de churn cercana al **50%** vs menos del 10% en clientes con más de un año. El onboarding es la ventana de máximo apalancamiento.

Implicancia: las primeras 12 semanas son donde un programa estructurado de bienvenida + seguimiento tiene

mayor retorno.

Hallazgo 3 — La regla "email a 15 días sin compra" no se sostiene

Los clientes que churnean ordenaron **más recientemente** que los activos (mediana 2 días vs 4 días). La regla histórica no tiene base empírica. El verdadero patrón de riesgo es **"compra reciente + queja sin resolver"** = 39% churn vs 7% en el escenario opuesto.

Implicancia: reorientar campañas de inactividad hacia atención one-on-one cuando aparece una queja en un cliente reciente.

Hallazgo 4 — La satisfacción auto-reportada no predice churn

Los clientes que se van reportan score **más alto** (3.4) que los que se quedan (3.0). El indicador puede estar inflado por sesgo de respuesta o medido en un momento que no captura intención.

Implicancia: no usar el score como gatillo de retención sin antes validar con el equipo de datos cómo y cuándo se recolecta.

10. Recomendaciones accionables

Cuatro acciones priorizadas (todas con verbo + objeto + impacto + responsable + plazo, alineadas al exec summary).

Nota sobre los números de impacto: los porcentajes "base" de cada acción (≥85% Recall del modelo en test; 39% en el segmento "nuevo + queja"; ~50% en clientes nuevos) son **observados en los datos** — del modelo o del análisis exploratorio. Los porcentajes "objetivo" (25%, 30%) son **targets aspiracionales para definir con el equipo comercial**, no proyecciones del modelo. El impacto real de cada intervención debe validarse con A/B testing en producción antes de comprometerse a la cifra. Lo que el modelo aporta es la **lista priorizada de clientes a contactar** — el lift real depende de qué tan buena sea la intervención.

Acción	Plazo	Responsable	Métrica de impacto
A. Reorientar campañas hacia tasa mensual de actividad	30 días	Marketing Retención	≥ 85% Recall en alertas (vs <50% con regla 15 días)
B. Atención one-on-one para nuevos (≤3m) con queja	Inmediato	Customer Success	Reducir churn del segmento de 39% a 25%
C. Programa estructurado de onboarding 90 días	90 días	CS + Producto	Reducir churn de nuevos de ~50% a ~30%
D. Re-evaluar el uso del score de satisfacción como gatillo	30 días	Analytics + Equipo de Datos	Ahorrar costo de campañas mal dirigidas

11. Limitaciones declaradas honestamente

1. **Variable Complain**: dataset público — no podemos confirmar el timing de la queja vs el churn. Postura conservadora: se dropeó del modelo. Si el equipo de datos confirma que es válida, recuperamos ~1% de Recall.
2. **Test set evaluado dos veces**: primero con RF defaults, después con RF V1 tras la iteración del tuning. Declarado como limitación. Recomendación: para producción a largo plazo, hacer un nuevo holdout 100% intocado para validación final.
3. **Generalización**: el modelo se entrenó con datos del año pasado. La efectividad puede degradarse a medida que cambia el mix de clientes. Re-entrenamiento **trimestral** mantiene la performance.
4. **Falsas alarmas**: ~42 clientes activos marcados como riesgo por lote de 223 alertas (~19%). Costo bajo por alarma (un email, un cupón) pero saturación del equipo comercial si no se gestiona.
5. **Causalidad vs correlación**: el modelo identifica patrones; no explica causalidad. "Clientes con baja tasa mensual churnean más" no significa que aumentar la tasa va a evitar el churn — pueden ser síntomas de un tercer factor. Las campañas dirigidas deberían medirse con A/B testing antes de escalar.
6. **Tuning leakage**: los hiperparámetros optimizan al CV, lo cual sesga ligeramente el estimador. Mitigado por CV repetido y validación del gap train-CV.

12. Próximos pasos

Priorizados por impacto esperado:

1. **Validar el timing de Complain con el equipo de datos** (semana 1). Es la mejora más barata: si se confirma legitimidad, recuperamos 1-2 puntos de Recall sin más esfuerzo.
2. **Activar la recomendación A (campañas por tasa mensual de actividad)** y medir en 30/60/90 días la diferencia de churn detectado vs la regla actual de los 15 días.
3. **A/B testing de las acciones B y C** (atención one-on-one en quejas de nuevos + onboarding estructurado de 90 días). Una rama recibe la intervención, otra el flujo actual. Medir churn a 90 días post-intervención.
4. **Re-entrenar el modelo trimestralmente** con datos frescos. Proceso fijo, no ad-hoc.
5. **Análisis profundo del segmento Single** (26.7% churn vs 16.8% base). Diseñar estrategia específica de fidelización para este perfil.

Apéndice A — Glosario técnico para la defensa oral

Los 13 términos de la rúbrica con definición + analogía + ejemplo del proyecto:

Término	Definición	Analogía	Ejemplo del proyecto
---------	------------	----------	----------------------

Churn	Cliente que deja de comprarnos	"Como un alumno que se borra del gimnasio sin avisar"	16.8% de la base = 945 clientes
Class imbalance	Una clase aparece mucho más que la otra	"Si decís que todos son diestros, acertás 90% sin haber aprendido nada"	83% activos / 17% churn
Leakage	Modelo usa info que en producción no estaría	"Estudiar con las respuestas del examen"	Variable Complain (timing no validable)
Train/test split	80% entrenar, 20% evaluar	"El examen sorpresa que no podés ver hasta el final"	4504 / 1126 clientes
Stratified	Split respeta proporción de clases	"Ambas porciones de la torta con mismo % de chocolate"	Train 16.83% churn / test 16.87%
Baseline / Dummy	Modelo más simple posible	"Tirar una moneda — el piso de comparación"	DummyClassifier: 83% accuracy, 0% Recall
Decision Tree	Preguntas sí/no en cascada	"Cuestionario fácil de leer"	DT: Recall 0.7652, obligatorio per rúbrica
Random Forest	Ensemble de cientos de árboles	"En vez de un médico, le preguntás a 500 y votan"	RF V1: Recall 0.9526 en test
Accuracy	% de aciertos total	"Engaña con desbalance — dummy da 83% sin servir"	No es métrica principal
Recall	De los que se van, ¿a cuántos detecto?	"El radar del hospital — no querés perderte ningún caso"	95.3% en test
Precision	De los que predigo churn, ¿cuántos son?	"De los que mando a tratamiento, ¿cuántos lo necesitaban?"	81.2% en test
SHAP values	Contribución de cada feature a cada predicción	"El ticket del super: no solo el total, sino qué producto sumó cuánto"	Global summary + 2 ejemplos locales
BayesSearchCV	Optimización de hiperparámetros bayesiana	"Aprende de las pruebas previas en vez de tirar dardos"	3 rondas iterativas

Apéndice B — Estructura del repo

churn_tp/	
├─ CLAUDE.md	Documentación principal del proyecto
├─ decisions.md	19 decisiones documentadas
├─ recursos_github.md	Referencias externas
├─ requirements.txt	Deps Python
├─ RESUMEN_EQUIPO.md	Handoff rápido para el equipo
├─ TP_Negocios_Consigna(1).pdf	
├─ .claude/	
└─ commands/	5 skills del TP (/add-decision, /ml-churn, etc.)
└─ skills/	11 skills del data-science-kit

	└─ data/	
	└─ raw/	CSV original
	└─ processed/	4 CSVs procesados + dataset_limpio
	└─ notebooks/	
	└─ 01_EDA_Churn.ipynb	EDA + 6 hipótesis
	└─ 01b_Investigacion_Anomalias_H3_H4.ipynb	Sub-investigaciones
	└─ 02_Preparacion_Datos.ipynb	Pipeline
	└─ 02b_Feature_Engineering_Exploracion.ipynb	Benchmark 7 candidatas
	└─ 03_Modeling_Churn.ipynb	Modelo final + tuning + SHAP
	└─ src/	
	└─ preprocessing.py	
	└─ models/	
	└─ __init__.py	
	└─ train.py	Script CLI reproducible
	└─ reports/	
	└─ 01_hipotesis.md	
	└─ handoff_to_modeler.md	
	└─ modeling_results.md	
	└─ runs_log.csv	
	└─ executive_summary.md / .pdf	Reporte ejecutivo (5 págs)
	└─ defense_executive.pdf	Deck ejecutivo (12 págs)
	└─ defense_technical.pdf	Deck técnico (19 págs)
	└─ talking_points.md	
	└─ glossary_defense.md	
	└─ design_brief.md	
	└─ resumen_proyecto.md / .pdf	Este documento
	└─ *.png	9 gráficos del proyecto
	└─ models/	Modelos .pkl (gitignored)

Documento generado para acompañar la defensa académica del Trabajo Práctico de Inteligencia Artificial Aplicada a Negocios. Toda la metodología, código, decisiones y datos están disponibles en el repositorio del proyecto.

--	--	--

--	--
